

PROGRAMACIÓN II

Trabajo Práctico 7: Herencia y Polimorfismo en Java

OBJETIVO GENERAL

Comprender y aplicar los conceptos de herencia y polimorfismo en la Programación Orientada a Objetos, reconociendo su importancia para la reutilización de código, la creación de jerarquías de clases y el diseño flexible de soluciones en Java.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Herencia	Uso de `extends` para crear jerarquías entre clases, aprovechando el principio is-a.
Modificadores de acceso	Uso de private, protected y public para controlar visibilidad.
Constructores y super	Invocación al constructor de la superclase con super(...) para inicializar atributos.
Upcasting	Generalización de objetos al tipo de la superclase.
Instanceof	Comprobación del tipo real de los objetos antes de hacer conversiones seguras.
Downcasting	Especialización de objetos desde una clase general a una más específica.
Clases abstractas	Uso de abstract para definir estructuras base que deben ser completadas por subclasses.

Métodos abstractos	Declaración de comportamientos que deben implementarse en las clases derivadas.
Polimorfismo	Uso de la sobrescritura de métodos (@Override) y llamada dinámica de métodos.
Herencia	Uso de `extends` para crear jerarquías entre clases, aprovechando el principio is-a.

1

Caso Práctico

Desarrollar las siguientes Katas en Java aplicando herencia y polimorfismo. Se recomienda repetir cada kata para afianzar el concepto.

1. Vehículos y herencia básica

- Clase base: Vehículo con atributos marca, modelo y método **mostrarInfo()**

```

public class Vehiculo {
    public String marca;
    public String modelo;

    // Constructor
    public Vehiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    public void mostrarInfo() {
        System.out.println("Marca: " + marca + ", Modelo: " + modelo);
    }
}

```

La clase **Vehiculo** es la que define lo que todo vehículo debe tener: sus **atributos marca y modelo**, y un método **mostrarInfo()** que imprime estos datos en pantalla. El constructor de la clase **recibe** como parámetros la **marca y el modelo**, y se encarga de inicializarlos, de modo que cada **objeto Vehiculo** que creemos tenga sus datos propios desde el momento de su creación. El método **mostrarInfo()** puede ser llamado desde cualquier objeto de esta clase para ver la información completa del vehículo.

- Subclase: Auto con atributo adicional **cantidadPuertas**, sobrescribe **mostrarInfo()**

```

public class Auto extends Vehiculo {
    public int cantidadPuertas;

    public Auto(String marca, String modelo, int cantidadPuertas) {
        super(marca, modelo); // llama al constructor de Vehiculo
        this.cantidadPuertas = cantidadPuertas;
    }

    Override
    public void mostrarInfo() {
        super.mostrarInfo();
        System.out.println("Cantidad de puertas: " + cantidadPuertas);
    }
}

```

La clase **Auto** es una **subclase de Vehiculo**, lo que significa que hereda los atributos marca y modelo, así como el método **mostrarInfo()** de la clase padre. Además, agrega un atributo propio llamado **cantidadPuertas** y define un constructor que recibe la marca, el modelo y la cantidad de puertas, utilizando `super(marca, modelo)` para inicializar los atributos heredados. El método **mostrarInfo()** se sobrescribe con `@Override` para primero mostrar la información de la **clase Vehiculo** y luego imprimir también la cantidad de puertas, permitiendo que un objeto Auto muestre toda su información completa de manera coherente.

- Tarea: Instanciar un auto y mostrar su información completa.

```

public class Principal {
    public static void main(String[] args) {
        Auto miAuto = new Auto("Toyota", "Corolla", 4);
        miAuto.mostrarInfo();
    }
}

```

```

Output - HerenciaPolimorfismo (run) x
run:
Marca: Toyota, Modelo: Corolla
Cantidad de puertas: 4
BUILD SUCCESSFUL (total time: 0 seconds)

```

Dentro de este método se crea un objeto Auto utilizando su constructor, pasando la marca, el modelo y la cantidad de puertas como parámetros. Luego se llama al método **mostrarInfo()** del objeto, lo que permite imprimir en pantalla toda la información del auto, incluyendo tanto los atributos heredados de Vehiculo como el atributo específico de Auto. Esta clase sirve para probar que la herencia y la sobrescritura de métodos funcionan correctamente.

2. Figuras geométricas y métodos abstractos

- Clase abstracta: Figura con método **calcularArea()** y atributo nombre

```

    * @author Sandra Martinez
    */
    public abstract class Figura {
        public String nombre;

        public Figura(String nombre) {
            this.nombre = nombre;
        }

        public abstract double calcularArea();

        public void mostrarArea() {
            System.out.println(nombre + " - Área: " + calcularArea());
        }
    }

```

La clase **Figura** es abstracta y no se pueden crear objetos directamente de ella. Tiene un atributo **nombre** y un método abstracto **calcularArea()** que obliga a las subclases a definir cómo se calcula el área. También tiene un método **mostrarArea()** que imprime el nombre de la figura y su área, usando el método **calcularArea()**.

- Subclases: **Círculo y Rectángulo** implementan el cálculo del área

```

    */
    public class Circulo extends Figura {
        public double radio;

        public Circulo(double radio) {
            super("Círculo");
            this.radio = radio;
        }

        Override
        public double calcularArea() {
            return Math.PI * radio * radio;
        }
    }

```

```

0  */
1  public class Rectangulo extends Figura {
2      public double ancho;
3      public double alto;
4
5      public Rectangulo(double ancho, double alto) {
6          super("Rectángulo");
7          this.ancho = ancho;
8          this.alto = alto;
9      }
10
11      Override
12      public double calcularArea() {
13          return ancho * alto;
14      }
15

```

Las subclases **Circulo** y **Rectangulo** son clases que **heredan de la clase abstracta Figura**, es decir, toman todo lo que tiene Figura y le agregan lo propio. Cada una tiene sus propios atributos: Circulo tiene radio y Rectangulo tiene ancho y alto. Además, cada subclase **implementa el método calcularArea()**, que es abstracto en Figura, y define cómo se calcula el área según la figura. Cuando creamos un objeto de **Circulo** o **Rectangulo**, podemos llamar al método **mostrarArea()** de la clase base y este llama automáticamente al cálculo correcto del área de la subclase. Así funciona el polimorfismo: usamos la misma función **mostrarArea()** pero cada figura calcula su área de manera diferente.

- Tarea: Crear un array de figuras y mostrar el área de cada una usando polimorfismo.

```
public class Principal {
    public static void main(String[] args) {
        Figura[] figuras = new Figura[2];
        figuras[0] = new Circulo(3);
        figuras[1] = new Rectangulo(4.0, 5.0);

        for (Figura f : figuras) {
            f.mostrarArea();
        }
    }
}
```

```
Output - HerenciaPolimorfismo (run) X
run:
Circulo - area: 28.274333882308138
Rectangulo - area: 20.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Dentro del **main**, se crea un array llamado **figuras** que puede guardar objetos del tipo **Figura** o de cualquier clase que herede de ella. Luego se crean dos objetos: uno de **Circulo** y otro de **Rectangulo**, que se guardan dentro del **array**. Después, con un bucle **for**, se recorre ese array y se llama al método **mostrarArea()** de cada figura. Aunque todas las posiciones del array son del tipo **Figura**, Java ejecuta el método correspondiente a cada subclase gracias al polimorfismo, mostrando el área calculada según la figura.

3. Empleados y polimorfismo

- Clase abstracta: Empleado con método **calcularSueldo()**

```
10  */
11  public abstract class Empleado {
12      protected String nombre;
13
14      public Empleado(String nombre) {
15          this.nombre = nombre;
16      }
17
18      public abstract double calcularSueldo();
19
20      public void mostrarSueldo() {
21          System.out.println(nombre + " - Sueldo: " + calcularSueldo());
22      }
23  }
```

- Subclases: **EmpleadoPlanta, EmpleadoTemporal**

```
0  */
1  public class EmpleadoTemporal extends Empleado {
2      private double horasTrabajadas;
3      private double valorHora;
4
5      public EmpleadoTemporal(String nombre, double horasTrabajadas, double valorHora) {
6          super(nombre);
7          this.horasTrabajadas = horasTrabajadas;
8          this.valorHora = valorHora;
9      }
10
11      Override
12      public double calcularSueldo() {
13          return horasTrabajadas * valorHora;
14      }
15  }
```

- Tarea: Crear lista de empleados, invocar **calcularSueldo()** polimórficamente, usar instanceof para clasificar

```

public class Principal {
    public static void main(String[] args) {
        ArrayList<Empleado> empleados = new ArrayList<>();

        empleados.add(new EmpleadoPlanta("Sandra", 50000));
        empleados.add(new EmpleadoTemporal("Oscar", 120, 2000));
        empleados.add(new EmpleadoPlanta("Leandro", 60000));
        empleados.add(new EmpleadoTemporal("María", 80, 2500));

        for (Empleado e : empleados) {
            e.mostrarSueldo();

            if (e instanceof EmpleadoPlanta) {
                System.out.println("Tipo: Planta");
            } else if (e instanceof EmpleadoTemporal) {
                System.out.println("Tipo: Temporal");
            }
            System.out.println("-----");
        }
    }
}

```

```

Output - HerenciaPolimorfismo (run) X
Run:
Sandra - Sueldo: 50000.0
Tipo: Planta
-----
Oscar - Sueldo: 240000.0
Tipo: Temporal
-----
Leandro - Sueldo: 60000.0
Tipo: Planta
-----
María - Sueldo: 200000.0
Tipo: Temporal
-----
BUILD SUCCESSFUL (total time: 0 seconds)

```

Primero, crea una lista de empleados (**ArrayList<Empleado>**) y agrega objetos de **EmpleadoPlanta** y **EmpleadoTemporal**. Luego recorre la lista con un bucle **for** y, para cada empleado, llama a **mostrarSueldo()**, que imprime el nombre y el sueldo usando polimorfismo. Después usa **instanceof** para identificar si el empleado es de planta o temporal y muestra su tipo. Entre cada empleado imprime una línea separadora para que la salida sea más clara.

4. Animales y comportamiento sobrescrito

- Clase: Animal con método **hacerSonido()** y **describirAnimal()**

```

public class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void hacerSonido() {
        System.out.println("El animal hace un sonido genérico");
    }

    public void describirAnimal() {
        System.out.println("Este es un animal llamado " + nombre);
    }
}

```

La clase **Animal** es la base de todos los animales. Tiene un atributo nombre para identificar al animal, un constructor que recibe el nombre, un método **hacerSonido()** que imprime un sonido genérico y se puede sobrescribir en las subclases, y un método **describirAnimal()** que imprime información básica sobre el animal.

- Subclases: Perro, Gato, Vaca sobrescriben **hacerSonido()** con **@Override**

```

public class Perro extends Animal {

    public Perro(String nombre) {
        super(nombre);
    }

    Override
    public void hacerSonido() {
        System.out.println(nombre + " hace: guauuuu");
    }
}

```

Perro: La clase Perro extiende a Animal y sobrescribe **hacerSonido()** usando **@Override** para imprimir el sonido característico del perro. Su constructor recibe el nombre y lo pasa a la clase padre con **super(nombre)**.

```

public class Gato extends Animal {

    public Gato(String nombre) {
        super(nombre);
    }

    Override
    public void hacerSonido() {
        System.out.println(nombre + " hace: miauuuu");
    }
}

```

Gato: La clase Gato extiende a Animal y sobrescribe **hacerSonido()** para imprimir el sonido del gato. Al igual que Perro, su constructor recibe el nombre y lo asigna usando **super(nombre)**.


```

public class Vaca extends Animal {

    public Vaca(String nombre) {
        super(nombre);
    }

    Override
    public void hacerSonido() {
        System.out.println(nombre + " hace: muuuu")
    }
}

```

Vaca: La clase Vaca extiende a Animal y sobrescribe **hacerSonido()** para imprimir el sonido de la vaca. Su constructor también recibe el nombre y lo pasa al padre con `super(nombre)`.

- Tarea: Crear lista de animales y mostrar sus sonidos con polimorfismo

```

* @author Sandra Martinez
*/
public class Principal {
    public static void main(String[] args) {
        ArrayList<Animal> animales = new ArrayList<>();

        animales.add(new Perro("Mike"));
        animales.add(new Gato("Rafael"));
        animales.add(new Vaca("Mora"));

        for (Animal a : animales) {
            a.hacerSonido();
            a.describirAnimal();
            System.out.println("-----");
        }
    }
}

```

```

run:
Mike hace: guauuuu
Este es un animal llamado Mike
-----
Rafael hace: miauuuu
Este es un animal llamado Rafael
-----
Mora hace: muuuu
Este es un animal llamado Mora
-----
BUILD SUCCESSFUL (total time: 0 seconds)

```

La clase **Principal** crea una lista de animales y agrega instancias de **Perro**, **Gato** y **Vaca**. Luego recorre la lista e invoca **hacerSonido()** y **describirAnimal()** de cada animal, mostrando sus sonidos y nombres en la consola. Gracias al polimorfismo, aunque la lista sea de tipo `Animal`, cada subclase ejecuta su propia versión de **hacerSonido()**, haciendo que la salida sea específica para cada animal.

CONCLUSIONES ESPERADAS

- Comprender el mecanismo de herencia y sus beneficios para la reutilización de código.

- Aplicar polimorfismo para lograr flexibilidad en el diseño de programas.
- Inicializar objetos correctamente usando **super** en constructores.
- Controlar el acceso a atributos y métodos con modificadores adecuados.
- Identificar y aplicar **upcasting**, **downcasting** y **instanceof** correctamente.
- Utilizar clases y métodos abstractos como base de jerarquías lógicas.
- Aplicar principios de diseño orientado a objetos en la implementación en Java.

La herencia permite reutilizar código creando clases base y subclases. El polimorfismo da flexibilidad, dejando que objetos distintos respondan a los mismos métodos de forma específica. **super** se usa para inicializar objetos correctamente en constructores de subclases. Los modificadores controlan el acceso a atributos y métodos. **upcasting**, **downcasting** y **instanceof** ayudan a manejar tipos y evitar errores. Las clases y métodos abstractos sirven como base de jerarquías lógicas, y todo esto aplica los principios de diseño orientado a objetos en Java.

Enlace Github:

<https://github.com/Sandra86Martinez/Tp7HerenciaPolimorfismo>